

// Code solution for Articulating Answers to Technical Questions Learning Activity

```
using System;
using System.Collections.Generic;

public class DuplicateFinder
{
    public static char FindFirstDuplicate(string text)
    {
        HashSet<char> lettersSeen = new HashSet<char>();

        foreach (char letter in text)
        {
            // Check if the letter was already seen
            if (lettersSeen.Contains(letter))
            {
                return letter;
            }

            // Add the letter to the set
            lettersSeen.Add(letter);
        }

        // Return '?' if there are no duplicates
        return '?';
    }

    public static void Run()
    {
        Console.WriteLine(FindFirstDuplicate("abcddef")); // d
        Console.WriteLine(FindFirstDuplicate("abcdef"));  // ?
    }
}
```

-- 1. What are some possible test cases for this problem?**

Some examples to test are:

- `"abcddef"` → the first repeated letter is `d`
- `"abcddeef"` → the first repeated letter is `d`
- `"abcdefde"` → the first repeated letter is `d`
- `"abcdef"` → no repeated letters
- `"aaaaaaaaa"` → the repeated letter is `a`

These test cases help make sure the program works correctly.

--2. What data structures could be useful? What is their performance?**

A -- Set -- is useful because it stores only unique values.
It is good for checking if a letter has already appeared.

Performance:

- Adding an item: $O(1)$
- Checking for an item: $O(1)$

A --Dictionary-- (or Map) could also be used if we want to count how many times each letter appears.

Performance:

- Adding items: $O(1)$
- Looking up items: $O(1)$

These structures are fast and work well for this type of problem.

--3. What boundary cases should be checked?**

Some important cases are:

- `""` → an empty string
- `"abc"` → no duplicates
- `"aaaaaaaa"` → all letters are the same
- `"abB"` → uppercase and lowercase letters
- `"ab2(:~"` → symbols and numbers
- `"nñ"` → special characters

These cases help test different situations the program may face.

--4. Describe one way to solve the problem.--

1. Create an empty `Set`.
2. Go through the string one letter at a time.
3. Check if the letter is already in the `Set`.
4. If it is already there, return that letter because it is the first duplicate.
5. If it is not there, add it to the `Set`.
6. If no duplicate letters are found, return `"?".`

This method is simple and works quickly.